Python Interview Preparation Guide

80+ Interview Q & A from Basics to Advanced

PYTHON BASICS

---

1. What is Python?

Python is a high-level interpreted language used for automation, data analysis, backend development, and scripting because it is easy to read and fast to develop with.

Use case: Writing ETL scripts, analysis notebooks, backend APIs.

```
print("Hello Python")
```

---

2. What does interpreted mean in Python?

Python executes code line by line at runtime, which makes debugging easier compared to compiled languages.

Use case: Faster debugging in analytics and scripting tasks.

```
x = 10
print(x)
```

---

3. What are variables in Python?

Variables store values in memory and are created automatically when a value is assigned.

Use case: Storing intermediate calculations or inputs.

```
age = 25
name = "Alice"
```

---

4. Is Python statically typed or dynamically typed?

Python is dynamically typed, meaning the variable type is decided at runtime and can change.

Use case: Rapid development without type declarations.

```
x = 10
x = "ten"
```

---

5. What are common Python data types?

Python provides built-in data types like int, float, string, list, tuple, set, dictionary, and boolean to store different kinds of data.

Use case: Choosing correct structure for performance and clarity.

```
num = 10
price = 9.5
text = "Data"
```

---

6. Difference between list and tuple?

A list is mutable (can be changed), while a tuple is immutable (cannot be changed).

Use case: Lists for dynamic data, tuples for fixed configurations.

```
lst = [1, 2, 3]
tup = (1, 2, 3)
```

---

7. What is mutability in Python?

Mutability means whether an object's value can be modified after creation.

Use case: Avoiding unexpected changes in shared objects.

```
lst = [1, 2]
lst.append(3)
```

---

8. What is a dictionary?

A dictionary stores data as key–value pairs and is commonly used for structured or JSON-like data.

Use case: API responses, configuration data.

```
emp = {"id": 1, "name": "John"}
```

---

9. What is type casting?

Type casting converts one data type into another when required for operations.

Use case: Converting user input to numeric values.

```
age = int("25")
```

---

10. What is None in Python?

None represents the absence of a value and is often used as a default or placeholder.

Use case: Initializing variables or handling missing data.

```
result = None
```

---

11. Difference between None and 0?

0 is a numeric value, while None means no value exists.

Use case: Distinguishing missing data from actual zero values.

```
x = None
y = 0
```

---

12. What are conditional statements?

Conditional statements control program flow based on conditions.

Use case: Business rules, validations.

```
if age >= 18:
    print("Eligible")
```

---

13. What is a for loop?

A for loop is used to iterate over a sequence like a list or range.

Use case: Processing datasets or lists.

```
for i in range(3):
    print(i)
```

---

14. Difference between for and while loop?

A for loop is used when iterations are known, while a while loop runs until a condition becomes false.

Use case: Repeated processing until a condition is met.

```
while x < 5:
    x += 1
```

---

15. What is break?

break exits the loop immediately when a condition is satisfied.

Use case: Stop processing once required result is found.

```
for i in range(5):
    if i == 3:
        break
```

---

16. What is continue?

continue skips the current iteration and moves to the next one.

Use case: Ignoring invalid records in a dataset.

```
for i in range(5):
    if i == 2:
        continue
```

---

17. What is pass?

pass is a placeholder used when a statement is syntactically required but no logic is written yet.

Use case: Creating function stubs.

```
def func():
    pass
```

---

18. What is input()?

input() takes user input as a string and usually needs type conversion.

Use case: Taking runtime inputs.

```
age = int(input("Enter age: "))
```

---

19. What is slicing?

Slicing extracts a portion of a string, list, or tuple.

Use case: Substring or sublist extraction.

```
s = "Python"
s[1:4]
```

---

20. What is indentation in Python?

Indentation defines code blocks and is mandatory in Python.

Use case: Control flow and readability.

```
if True:
    print("Indented block")
```

INTERMEDIATE PYTHON

---

21. What is a function in Python?

A function is a reusable block of code that performs a specific task and helps reduce repetition and improve readability.

Use case: Writing reusable logic like calculations, validations, or data processing.

```
def add(a, b):
    return a + b

add(3, 4)
```

---

22. What are parameters and arguments?

Parameters are variables defined in a function, and arguments are the actual values passed during the function call.

Use case: Passing dynamic data to functions.

```
def greet(name):   # parameter
    print(name)

greet("Alice")     # argument
```

---

23. What are default arguments?

Default arguments allow a function to use a predefined value if no argument is passed.

Use case: Optional inputs in functions.

```
def greet(name="User"):
    print(name)

greet()
greet("John")
```

---

*24. What is *args in Python?*

*args allows a function to accept a variable number of positional arguments.

Use case: When the number of inputs is not fixed.

```
def total(*args):
    return sum(args)

total(1, 2, 3, 4)
```

---

25. What is **kwargs in Python?

**kwargs allows a function to accept a variable number of keyword arguments as a dictionary.

Use case: Passing configuration or named parameters.

```
def emp_info(**kwargs):
    print(kwargs)

emp_info(name="John", age=30)
```

---

26. What is a lambda function?

A lambda function is a small anonymous function written in one line for short operations.

Use case: Quick transformations in data processing.

```
square = lambda x: x * x
square(5)
```

---

27. What is map() function?

map() applies a function to each element of an iterable and returns a map object.

Use case: Applying transformations to lists.

```
nums = [1, 2, 3]
list(map(lambda x: x * 2, nums))
```

---

28. What is filter() function?

filter() selects elements from an iterable that satisfy a condition.

Use case: Filtering valid records from data.

```
nums = [2, 5, 8, 3]
list(filter(lambda x: x > 4, nums))
```

---

29. What is reduce() function?

reduce() applies a function cumulatively to elements and reduces them to a single value.

Use case: Aggregations like sum or product.

```
from functools import reduce
reduce(lambda a, b: a + b, [1, 2, 3, 4])
```

---

30. What is list comprehension?

List comprehension is a concise way to create lists using expressions and loops.

Use case: Cleaner and faster list creation.

```
squares = [x*x for x in range(5)]
```

---

31. What is set comprehension?

Set comprehension creates a set using an expression and automatically removes duplicates.

Use case: Extracting unique values.

```
unique_nums = {x for x in [1, 2, 2, 3, 3]}
```

---

32. What is dictionary comprehension?

Dictionary comprehension creates dictionaries in a compact way.

Use case: Transforming key-value data.

```
squares = {x: x*x for x in range(4)}
```

---

33. What is exception handling?

Exception handling allows programs to handle runtime errors without crashing.

Use case: Handling invalid input or runtime failures.

```
try:
    x = 10 / 0
except ZeroDivisionError:
    print("Cannot divide by zero")
```

---

34. What is the finally block?

The finally block always executes, whether an exception occurs or not.

Use case: Closing files or releasing resources.

```
try:
    f = open("data.txt")
finally:
    f.close()
```

---

35. What is a custom exception?

A custom exception is a user-defined error created to handle specific conditions.

Use case: Enforcing business rules.

```
raise ValueError("Invalid age")
```

---

36. How do you read a file in Python?

Files are read using open() and file methods like read(), readlines(), or iteration.

Use case: Reading logs or datasets.

```
with open("file.txt", "r") as f:
    data = f.read()
```

---

37. How do you write to a file in Python?

Writing to a file uses write mode which overwrites existing content.

Use case: Storing output or logs.

```
with open("file.txt", "w") as f:
    f.write("Hello")
```

---

38. What is the difference between write and append mode?

Write mode overwrites the file, while append mode adds content to the end.

Use case: Logging new entries without deleting old ones.

```
with open("file.txt", "a") as f:
    f.write("New line")
```

---

39. What is a module in Python?

A module is a file containing reusable Python code.

Use case: Code reuse and organization.

```
import math
math.sqrt(16)
```

---

40. What is name == "main"?

It ensures that code runs only when the file is executed directly, not when imported.

Use case: Separating reusable code from executable code.

```
if __name__ == "__main__":
    print("Run directly")
```

ADVANCED PYTHON

---

41. What is OOP in Python?

Object-Oriented Programming in Python organizes code using classes and objects to improve reusability, scalability, and structure.

Use case: Large applications, reusable business logic.

```
class Employee:
    pass
```

---

42. What is a class and an object?

A class is a blueprint, and an object is an instance created from that blueprint.

Use case: Representing real-world entities like users or products.

```
class Car:
    pass

c = Car()
```

---

43. What is init method?

init initializes object attributes when an object is created.

Use case: Assigning initial values to objects.

```
class Emp:
    def __init__(self, name):
        self.name = name

e = Emp("John")
```

---

44. What is inheritance?

Inheritance allows a class to reuse properties and methods of another class.

Use case: Sharing common logic across related classes.

```
class Manager(Emp):
    pass
```

---

45. What is polymorphism?

Polymorphism allows the same method name to behave differently based on context.

Use case: Flexible function behavior.

```
def add(a, b, c=0):
    return a + b + c
```

---

46. What is encapsulation?

Encapsulation restricts direct access to variables to protect data.

Use case: Preventing accidental data modification.

```
class Emp:
    def __init__(self):
        self.__salary = 50000
```

---

47. What is abstraction?

Abstraction hides implementation details and exposes only essential features.

Use case: Designing clean APIs.

```
from abc import ABC, abstractmethod

class Shape(ABC):
    @abstractmethod
    def area(self):
        pass
```

---

48. What are generators in Python?

Generators produce values one at a time using yield, saving memory.

Use case: Processing large datasets.

```
def gen():
    for i in range(3):
        yield i
```

---

49. Difference between yield and return?

yield pauses function execution and resumes later, while return ends the function.

Use case: Streaming data processing.

```
def numbers():
    yield 1
    yield 2
```

---

50. What are decorators?

Decorators modify the behavior of functions without changing their code.

Use case: Logging, authentication, validation.

```
def my_decorator(func):
    def wrapper():
        print("Before")
        func()
    return wrapper
```

---

51. What is a context manager?

A context manager automatically handles resource setup and cleanup.

Use case: File handling, database connections.

```
with open("file.txt") as f:
    data = f.read()
```

---

52. How does memory management work in Python?

Python manages memory automatically using reference counting and garbage collection.

Use case: Preventing memory leaks.

```
import gc
gc.collect()
```

---

53. What is GIL (Global Interpreter Lock)?

GIL allows only one thread to execute Python bytecode at a time.

Use case: CPU-bound multithreading limitations.

```
# Affects threading performance
```

---

54. Difference between multithreading and multiprocessing?

Multithreading shares memory, multiprocessing runs separate processes.

Use case: I/O-bound vs CPU-bound tasks.

```
from threading import Thread
from multiprocessing import Process
```

---

55. What is shallow copy?

Shallow copy copies references of nested objects.

Use case: Fast copy when deep structure is not needed.

```
import copy
copy.copy(lst)
```

---

56. What is deep copy?

Deep copy creates completely independent copies of objects.

Use case: Avoid shared references in nested objects.

```
copy.deepcopy(lst)
```

---

57. How do you handle JSON in Python?

Python uses the json module to parse and serialize JSON data.

Use case: Working with APIs.

```
import json
json.loads('{"a":1}')
```

---

58. What is regular expression (regex)?

Regex is used for pattern matching and text searching.

Use case: Data validation, log parsing.

```
import re
re.findall(r"\d+", "Order123")
```

---

59. What is a virtual environment?

A virtual environment isolates project dependencies.

Use case: Avoiding package conflicts.

```
python -m venv env
```

---

60. What is time complexity?

Time complexity measures how an algorithm's runtime grows with input size.

Use case: Writing efficient code.

```
# O(n) example
for i in range(n):
    pass
```

SCENARIO-BASED PYTHON

---

61. Reverse a string

This is done using slicing, which is the most Pythonic and efficient way.

Use case: String manipulation, interview warm-up questions.

```
s = "python"
rev = s[::-1]
```

---

62. Check if a string is a palindrome

A string is a palindrome if it reads the same forward and backward.

Use case: Text validation problems.

```
s = "madam"
is_palindrome = s == s[::-1]
```

---

63. Find duplicate elements in a list

Duplicates can be identified by counting occurrences.

Use case: Data cleaning and validation.

```
lst = [1,2,2,3,3,4]
dups = [x for x in set(lst) if lst.count(x) > 1]
```

---

64. Remove duplicates from a list while preserving order

Using a loop ensures order is maintained.

Use case: Cleaning user or transaction data.

```
lst = [1,2,2,3]
res = []
for x in lst:
    if x not in res:
        res.append(x)
```

---

65. Count frequency of words in a sentence

Using Counter simplifies frequency calculation.

Use case: Text analytics, NLP preprocessing.

```
from collections import Counter
Counter("data science data".split())
```

---

66. Find the maximum and minimum in a list

Built-in functions provide an efficient solution.

Use case: Finding extremes in datasets.

```
lst = [4,7,1,9]
max(lst), min(lst)
```

---

67. Find the second largest number in a list

Sorting and indexing is a simple approach.

Use case: Ranking problems.

```
lst = [10,20,30,40]
sorted(lst)[-2]
```

---

68. Find missing number from 1 to n

Uses the mathematical sum formula.

Use case: Data completeness checks.

```
lst = [1,2,4,5]
n = 5
missing = n*(n+1)//2 - sum(lst)
```

---

69. Check if a number is prime

A prime number has no divisors other than 1 and itself.

Use case: Mathematical validations.

```
n = 7
is_prime = all(n % i != 0 for i in range(2, n))
```

---

70. Generate Fibonacci series

Uses iteration to generate a sequence.

Use case: Algorithm fundamentals.

```
a, b = 0, 1
for _ in range(5):
    print(a)
    a, b = b, a+b
```

---

71. Sort a dictionary by value

Sorting is done using the key parameter.

Use case: Ranking metrics.

```
d = {"a":3,"b":1,"c":2}
sorted(d.items(), key=lambda x: x[1])
```

---

72. Merge two dictionaries

Python allows dictionary unpacking.

Use case: Combining configurations.

```
d1 = {"a":1}
d2 = {"b":2}
merged = {**d1, **d2}
```

---

73. Flatten a nested list

List comprehension helps flatten lists.

Use case: Data preprocessing.

```
lst = [[1,2],[3,4]]
flat = [i for sub in lst for i in sub]
```

---

74. Count vowels in a string

Checks each character against vowel set.

Use case: Text processing.

```
s = "python"
count = sum(1 for c in s if c in "aeiou")
```

---

75. Check if two strings are anagrams

Anagrams have the same sorted characters.

Use case: String comparison tasks.

```
a, b = "listen", "silent"
sorted(a) == sorted(b)
```

---

76. Find factorial of a number

Factorial can be computed using math module.

Use case: Mathematical operations.

```
import math
math.factorial(5)
```

---

77. Read a large file efficiently

Iterating line by line avoids memory issues.

Use case: Log file processing.

```
with open("file.txt") as f:
    for line in f:
        pass
```

---

78. Write data to a file

Used to store outputs or logs.

Use case: Persisting results.

```
with open("out.txt","w") as f:
    f.write("Hello")
```

---

79. Count number of lines in a file

Iterating and counting lines.

Use case: File statistics.

```
sum(1 for _ in open("file.txt"))
```

---

80. Handle missing values in a list

Filtering removes invalid entries.

Use case: Data cleaning.

```
lst = [1,None,3]
clean = [x for x in lst if x is not None]
```

---

81. Convert list of strings to integers

Uses map with int conversion.

Use case: Processing user inputs.

```
lst = ["1","2","3"]
nums = list(map(int, lst))
```

---

82. Find common elements between two lists

Set intersection finds common values.

Use case: Matching datasets.

```
set(a) & set(b)
```

---

83. Remove empty strings from list

Filters falsy values.

Use case: Cleaning text data.

```
lst = ["a","", "b"]
[x for x in lst if x]
```

---

84. Check if all elements are unique

Compare length of list and set.

Use case: Uniqueness validation.

```
len(lst) == len(set(lst))
```

---

85. Find cumulative sum of a list

Uses loop to maintain running total.

Use case: Trend analysis.

```
res = []
total = 0
for x in lst:
    total += x
    res.append(total)
```

---

86. API call using Python

requests library is commonly used.

Use case: Fetching external data.

```
import requests
requests.get("https://api.example.com")
```

---

87. Handle exceptions in API call

Use try-except to avoid crashes.

Use case: Robust applications.

```
try:
    requests.get(url)
except Exception as e:
    print(e)
```

---

88. Parse JSON response
json module converts JSON to dict.

Use case: API data handling.

```
import json
json.loads('{"id":1}')
```

---

89. Log errors to a file
Logging module is used.

Use case: Production debugging.

```
import logging
logging.error("Error occurred")
```

---

90. Measure execution time of code
Uses time module.

Use case: Performance testing.

```
import time
start = time.time()
```

---

91. Find index of an element in list
index() returns position.

Use case: Searching operations.

```
lst.index(5)
```

---

92. Split a string into words
Uses split() method.

Use case: Text parsing.

```
"text data".split()
```

---

93. Join list into string
join() combines elements.

Use case: Formatting output.

```
",".join(["a","b","c"])
```

---

94. Check data type of variable
Uses type() or isinstance().

Use case: Type validation.

```
isinstance(5, int)
```

---

95. Convert tuple to list
Uses list() constructor.

Use case: Modifying immutable data.

```
list((1,2,3))
```

---

96. Sort list in descending order
Uses sort with reverse.

Use case: Ranking.

```
lst.sort(reverse=True)
```

---

97. Find longest word in a sentence
Uses max with key.

Use case: Text analytics.

```
max("data science rocks".split(), key=len)
```

---

98. Check if string contains only digits
isdigit() validates numeric strings.

Use case: Input validation.

```
"123".isdigit()
```

---

99. Convert list to set and back to list

Used to remove duplicates.

Use case: Data deduplication.

```
list(set(lst))
```

---

100. Count characters in a string

Uses Counter for frequency.

Use case: Text analysis.

```
from collections import Counter
Counter("python")
```